

Explorando Algoritmos de Compressão de Dados: Teoria, Implementação e Desempenho

Gustavo Yujii Silva Kadooka

Orientadora: Profa. Dra. Andréa Carla Gonçalves Vianna

Universidade Estadual Paulista
Faculdade de Ciências - Câmpus Bauru
Ciência da Computação

Outubro de 2025

- 1 Introdução
- 2 Fundamentação Teórica
- 3 Implementação
- 4 Análise Comparativa
- 5 Conclusão

- A compressão de dados é essencial na ciência da computação moderna
- Reduz espaço de armazenamento e otimiza transmissão de dados
- Desenvolvimento iniciou na década de 1950 com o algoritmo de Huffman
- Algoritmos clássicos continuam sendo base de sistemas atuais

Algoritmos Clássicos

- **Huffman** (1952): Codificação de prefixo variável
- **LZ77** (1977): Compressão baseada em dicionário
- **LZW** (1984): Evolução do LZ77
- **GZIP**: Combinação de LZ77 e Huffman

Objetivos

Objetivo Geral

Investigar e comparar o desempenho de algoritmos clássicos de compressão de dados sem perdas, por meio da implementação prática dessas técnicas, aplicando-as a diferentes tipos de arquivos e analisando sua eficiência.

Objetivos Específicos

- Estudar fundamentos teóricos dos algoritmos selecionados
- Implementar em C++ os algoritmos para arquivos .txt, .bmp e .wav
- Desenvolver interface gráfica com GTK
- Criar scripts Python para análise comparativa
- Avaliar pontos fortes e fracos de cada algoritmo
- Identificar o algoritmo mais adequado para cada cenário

Entropia de Shannon (1948)

Medida da quantidade média de informação em uma fonte de dados:

$$H(X) = - \sum_{i=1}^n P(x_i) \log_2 P(x_i)$$

- Define o **limite teórico mínimo** para compressão sem perdas
- Quanto maior a entropia, menor a compressibilidade
- Baseado na quantificação de informação de Hartley (1928)

Exemplo Prático

Um arquivo com padrões repetitivos tem baixa entropia e alta compressibilidade. Dados aleatórios têm alta entropia e baixa compressibilidade.

Entropia: Exemplo da Moeda

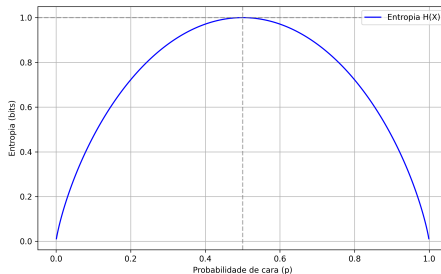


Figura 1: Entropia de uma moeda em função da probabilidade p

- Máxima entropia quando $p = 0,5$ (1 bit)
- Mínima entropia quando $p = 0$ ou $p = 1$ (0 bits)
- Princípio fundamental: incerteza gera informação

Redundância e Taxa de Compressão

Redundância

$$R = L - H(X)$$

onde L é o comprimento médio de codificação e $H(X)$ é a entropia.

Taxa de Compressão

$$\text{Taxa} = \frac{T_{\text{original}}}{T_{\text{comprimido}}}$$

$$\text{Redução} = \left(1 - \frac{T_{\text{comprimido}}}{T_{\text{original}}} \right) \times 100\%$$

Objetivo da compressão: Reduzir R aproximando L de $H(X)$

Algoritmo de Huffman

Princípio:

- Codificação estatística
- Códigos de comprimento variável
- Símbolos frequentes → códigos curtos
- Símbolos raros → códigos longos

Propriedades:

- Ótimo para símbolos independentes
- $H(X) \leq L < H(X) + 1$
- Código de prefixo válido

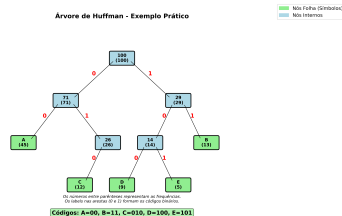


Figura 2: Exemplo de Árvore de Huffman

Algoritmo de Huffman: Exemplo Prático

Etapas do algoritmo

- Determinar a frequência de cada símbolo
- Montar uma árvore de Huffman
- Atribuir códigos aos símbolos, percorrendo a árvore da raiz até as folhas

Algoritmo de Huffman: Exemplo Prático

Determinar a frequência de cada símbolo

ciencia da compuacao

- c - 4
- i - 2
- e - 1
- n - 1
- a - 4
- d - 1
- o - 2
- m - 1
- p - 1
- u - 1
- ' ' - 2

Algoritmo de Huffman: Exemplo Prático

Ordenar

ciencia da computacao

- m - 1
- p - 1
- u - 1
- e - 1
- n - 1
- d - 1
- i - 2
- o - 2
- ' ' - 2
- c - 4
- a - 4

Algoritmo de Huffman: Exemplo Prático

m1 p1 u1 e1 n1 d1 i2 o2 ' 2 c4 a4

Figura 3: Exemplo de Árvore de Huffman

Algoritmo de Huffman: Exemplo Prático

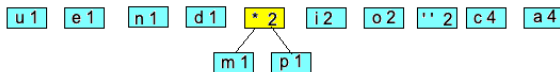


Figura 4: Exemplo de Árvore de Huffman

Algoritmo de Huffman: Exemplo Prático

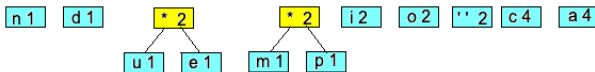


Figura 5: Exemplo de Árvore de Huffman

Algoritmo de Huffman: Exemplo Prático

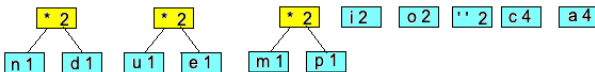


Figura 6: Exemplo de Árvore de Huffman

Algoritmo de Huffman: Exemplo Prático

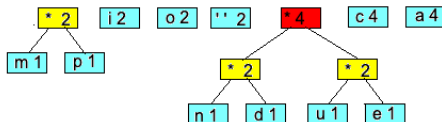


Figura 7: Exemplo de Árvore de Huffman

Algoritmo de Huffman: Exemplo Prático

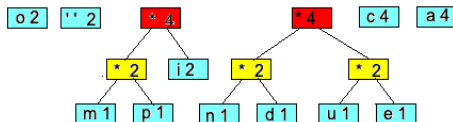


Figura 8: Exemplo de Árvore de Huffman

Algoritmo de Huffman: Exemplo Prático

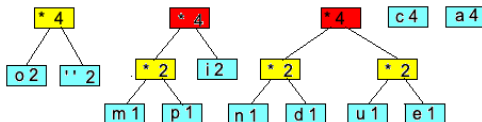


Figura 9: Exemplo de Árvore de Huffman

Algoritmo de Huffman: Exemplo Prático

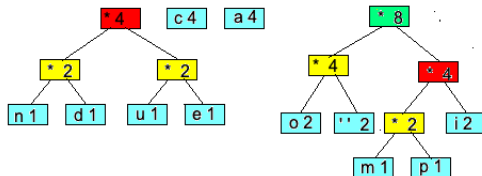


Figura 10: Exemplo de Árvore de Huffman

Algoritmo de Huffman: Exemplo Prático

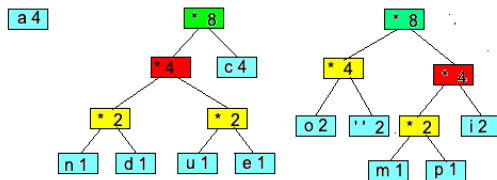


Figura 11: Exemplo de Árvore de Huffman

Algoritmo de Huffman: Exemplo Prático

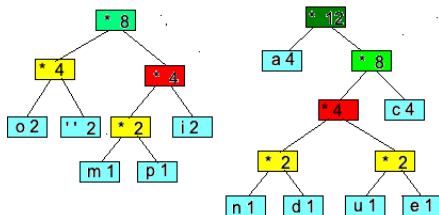


Figura 12: Exemplo de Árvore de Huffman

Algoritmo de Huffman: Exemplo Prático

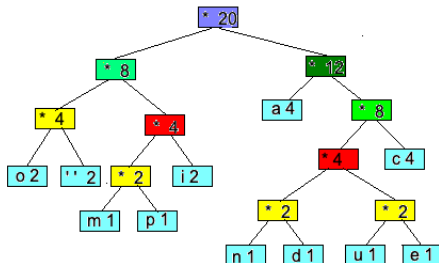


Figura 13: Exemplo de Árvore de Huffman

Algoritmo de Huffman: Exemplo Prático

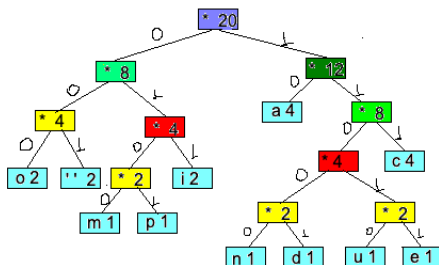


Figura 14: Exemplo de Árvore de Huffman

Algoritmo de Huffman: Exemplo Prático

Atribuir códigos aos símbolos, percorrendo a árvore da raiz até as folhas

- o - 000
- ' ' - 001
- m - 0100
- p - 0101
- i - 011
- a - 10
- n - 11000
- d - 11001
- c - 111
- u - 11010
- e - 11011

Algoritmo de Huffman: Exemplo Prático

Original

Considerando a tabela ASCII: $20 * 8 = 160$ bits

Algoritmo de Huffman: Exemplo Prático

Comprimido

Considerando a tabela de códigos de Huffman:

Símbolo	Código	Ocorrências	Bits
o	000	2	6
' '	001	2	6
m	0100	1	4
p	0101	1	4
i	011	2	6
a	10	4	8
n	11000	1	5
d	11001	1	5
c	111	4	12
u	11010	1	5
e	11011	1	5
Total			66

Princípio

Compressão baseada em dicionário com janela deslizante

Componentes:

- **Buffer de busca:** Dados já processados (4KB - 32KB)
- **Buffer de antecipação:** Dados a processar (4 - 258 bytes)
- **Codificação:** Triplo (distância, comprimento, próximo_símbolo)

Exemplo

Entrada: AACAAACABCABAAAC

Saída: (0,0,A); (1,1,C); (3,4,B); (3,3,A); (1,2,C)

Vantagem: Explora redundância temporal, não apenas estatística

Algoritmo LZ77

Assumindo uma janela de busca de tamanho 6 e buffer de antecipação de tamanho 4, o processo de compressão pode ser acompanhado na tabela 3.

Tabela 3 – Exemplo de funcionamento do algoritmo LZ77.

Passo	Buffer de busca	Buffer antecipação	Saída
1	(vazio)	AACA	(0,0,A)
2	A	ACAA	(1,1,C)
3	AAC	AACA	(3,4,B)
4	CAACAB	CABA	(3,3,A)
5	ABCABA	AAC	(1,2,C)

Fonte: Elaborada pelo autor

A saída comprimida seria:

(0,0,A); (1,1,C); (3,4,B); (3,3,A); (1,2,C)

Figura 15: Exemplo de funcionamento do algoritmo LZ77

LZW (1984)

- Evolução do LZ77
- Dicionário dinâmico
- Códigos de comprimento variável
- Usado em GIF e TIFF

GZIP

- Combina LZ77 + Huffman
- Algoritmo DEFLATE
- Padrão em sistemas Unix/Linux
- Usado em HTTP/HTTPS

Diferença do LZ77:

- Constrói dicionário incrementalmente
- Não usa janela deslizante
- Mais simples de implementar

Vantagens:

- Alta taxa de compressão
- Descompressão rápida
- Metadados de integridade (CRC32)

Características

- Linguagem: C++ (algoritmos) + Python (análise)
- Interface gráfica: GTK
- Formatos suportados: .txt, .bmp (24 bits), .wav (PCM)
- Algoritmos: Huffman (3 variantes), LZ77, LZW, GZIP

Arquitetura Modular

- Classes para manipulação de formatos (BMP, WAV)
- Classes base abstratas para algoritmos
- Classes especializadas por tipo de arquivo
- Scripts Python para geração de gráficos

Organização de Diretórios

- **include/** - Cabeçalhos (.hpp)
 - ▶ formats/ - Classes BMP e WAV
 - ▶ huffman/, lz77/, lzw/, gzip/ - Algoritmos de compressão
 - ▶ GUI.hpp - Interface gráfica
- **src/** - Implementações (.cpp)
 - ▶ Implementações dos algoritmos
 - ▶ python_graphs/ - Scripts para análise
 - ▶ main.cpp - Programa principal
- **data/**
 - ▶ input/ - 300 arquivos de teste (100 .txt, 100 .bmp, 100 .wav)
 - ▶ output/ - Resultados por algoritmo

Manipulação de Headers

Basic BMP File Format		
Name	Size	Description
Header	14 bytes	Windows Structure: BITMAPFILEHEADER
Signature	2 bytes	'BM'
FileSize	4 bytes	File size in bytes
reserved	4 bytes	unused (=0)
DataOffset	4 bytes	File offset to Raster Data
InfoHeader	40 bytes	Windows Structure: BITMAPINFOHEADER
Size	4 bytes	Size of InfoHeader = 40
Width	4 bytes	Bitmap Width
Height	4 bytes	Bitmap Height
Planes	2 bytes	Number of Planes (=1)
BitCount	2 bytes	Bits per Pixel 1 = monochrome palette, NumColors = 1 4 = 4bit palletized, NumColors = 16 8 = 8bit palletized, NumColors = 256 16 = 16bit RGB, NumColors = 65536 (?) 24 = 24bit RGB, NumColors = 16M
Compression	4 bytes	Type of Compression 0 = BI_RGB no compression 1 = BI_RLE8 8bit RLE encoding 2 = BI_RLE4 4bit RLE encoding
ImageSize	4 bytes	(compressed) Size of Image It is valid to set this = 0 if Compression = 0
XpixelsPerM	4 bytes	horizontal resolution: Pixels/meter
YpixelsPerM	4 bytes	vertical resolution: Pixels/meter
ColorsUsed	4 bytes	Number of actually used colors
ColorsImportant	4 bytes	Number of important colors 0 = all
ColorTable	4 * NumColors bytes	present only if Info.BitsPerPixel <= 8 colors should be ordered by importance
Red	1 byte	Red intensity
Green	1 byte	Green intensity
Blue	1 byte	Blue intensity
reserved	1 byte	unused (=0)
repeated NumColors times		
Raster Data	Info ImageSize bytes	The pixel data

Figura 16: Estrutura do header BMP

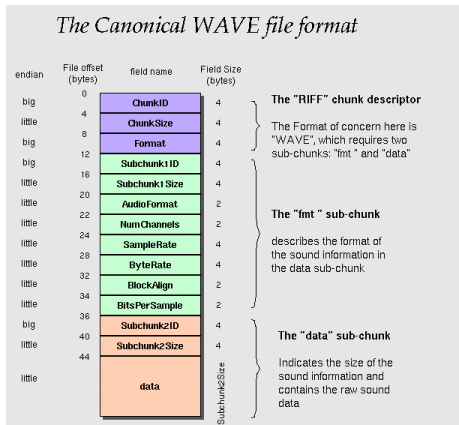


Figura 17: Estrutura do header WAVE

Implementação: Classe BMP

```
1  #pragma pack(push,1)
2  class bmp {
3  public:
4      struct HEADER {
5          char signature[2];
6          uint32_t file_size;
7          uint32_t reserved;
8          uint32_t data_offset;
9      } header;
10
11     struct INFO_header {
12         uint32_t size;
13         uint32_t width, height;
14         uint16_t planes, bits_per_pixel;
15         uint32_t compression, image_size;
16         // ... outros campos
17     } info_header;
18
19     uint8_t *color_table;
20     uint8_t *data;
21     size_t data_size;
22
23     int read(const char* filename);
24     void print();
25     uint32_t get_color_table_size();
26     int has_color_table();
27 };
28 #pragma pack(pop)
```

Classe BitStream

Problema

Linguagens como C/C++ são orientadas a bytes. Algoritmos de compressão precisam manipular bits individuais para máxima eficiência.

Solução: Classes BitWriter e BitReader

- **BitWriter:** Acumula bits em buffer de 32 bits antes de escrever
- **BitReader:** Lê bytes e extrai bits individuais
- Otimiza operações de I/O
- Essencial para LZW e LZ77

Importância

Sem manipulação bit-a-bit, seria impossível atingir as taxas de compressão teóricas previstas pela entropia.

① Huffman 1 (Tree on memory):

- ▶ Árvore mantida em memória durante compressão
- ▶ Códigos gerados diretamente da árvore

② Huffman 2 (Tree on file):

- ▶ Árvore salva junto com dados comprimidos
- ▶ Permite reconstrução durante descompressão

③ Huffman 3 (Canonical coding):

- ▶ Códigos canônicos ordenados
- ▶ Otimiza armazenamento de metadados
- ▶ Reduz overhead do dicionário

Estrutura de dados customizada: Lista encadeada ordenada em vez de priority queue

Interface Gráfica



Figura 18: Tela principal do GKcompress

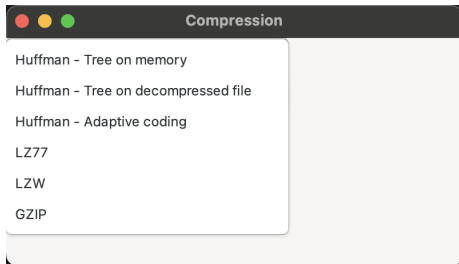


Figura 19: Tela de processamento

- Desenvolvida com GTK
- Seleção de algoritmo e arquivo
- Feedback visual do progresso
- Demonstração em vídeo no YouTube

Conjunto de Dados

- **300 arquivos** sintéticos gerados
- 100 arquivos .txt (tamanho médio: 33 KB)
- 100 arquivos .bmp (tamanho médio: 264 KB)
- 100 arquivos .wav (tamanho médio: 109 KB)

Geração Controlada de Entropia

- **Baixa entropia:** Padrões simples, alta repetição
- **Média entropia:** Dados estruturados com variação
- **Alta entropia:** Dados pseudo-aleatórios

Justificativa: Dados sintéticos permitem controlar características estatísticas e testar limites dos algoritmos

Resultados: Arquivos de Texto

Table 1: Taxa média de compressão para arquivos TXT

Algoritmo	Taxa (%)	Desvio Padrão
Huffman 1	42,7	1,0
Huffman 2	42,4	1,0
Huffman 3	42,5	0,9
LZ77	46,1	7,4
LZW	49,4	1,9
GZIP	66,9	5,1

Destaque

GZIP obteve a melhor performance, explorando tanto redundância estatística quanto estrutural dos dados textuais.

Resultados: Arquivos de Imagem

Table 2: Taxa média de compressão para arquivos BMP

Algoritmo	Taxa (%)	Desvio Padrão
Huffman 1	40,7	45,9
Huffman 2	40,2	46,1
Huffman 3	40,4	46,0
LZ77	63,0	47,3
LZW	49,4	55,0
GZIP	70,6	43,5

Observação

LZ77 e GZIP apresentaram excelente desempenho devido à redundância espacial nas imagens bitmap geradas.

Resultados: Arquivos de Áudio

Table 3: Taxa média de compressão para arquivos WAV

Algoritmo	Taxa (%)	Desvio Padrão
Huffman 1	3,9	6,4
Huffman 2	3,4	6,4
Huffman 3	3,6	6,4
LZ77	39,8	51,0
LZW	6,2	43,0
GZIP	62,9	41,1

Desafio

Arquivos de áudio apresentaram maior variabilidade nos resultados devido à natureza do sinal, especialmente em ruído branco e sinais complexos.

Análise Visual: Comparação TXT

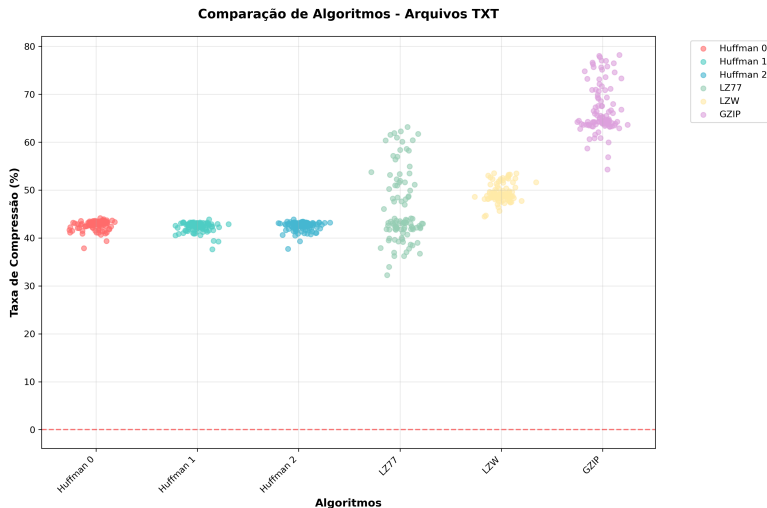


Figura 20: Scatter plot dos resultados para arquivos TXT

Análise Visual: Comparação BMP

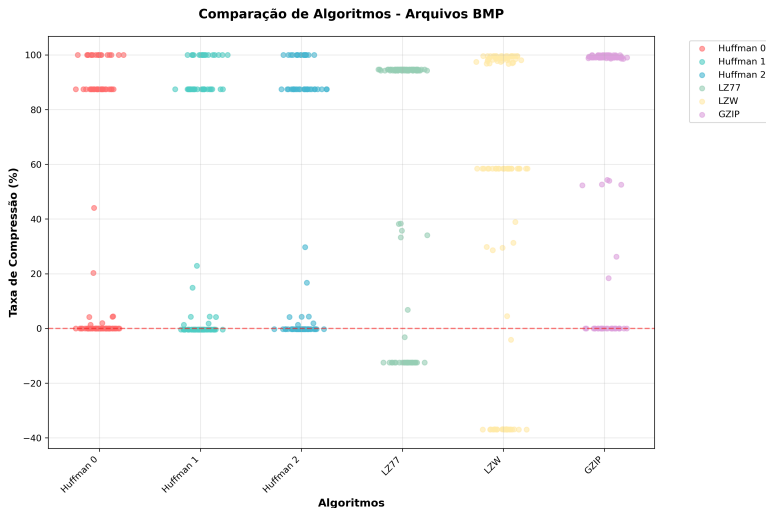


Figura 21: Resultados de compressão para arquivos BMP

Análise Visual: Comparação WAV

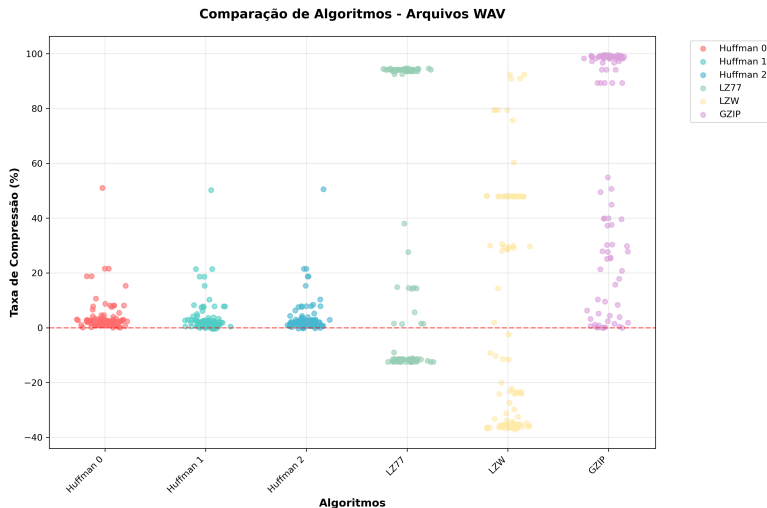


Figura 22: Scatter plot dos resultados para arquivos WAV

Teste ANOVA

- Aplicado para cada tipo de arquivo
- Resultados: $p < 0,001$ para todos os tipos
- **Conclusão:** Diferenças entre algoritmos são estatisticamente significativas

Correlação com Tamanho do Arquivo

- **TXT:** Correlação positiva moderada ($r = 0,45$)
- **BMP:** Correlação negativa fraca ($r = -0,23$)
- **WAV:** Correlação negativa moderada ($r = -0,38$)

Implicação: A relação entre tamanho e compressão varia por tipo de dado

A Entropia como Fator Determinante

Validação Empírica

Os resultados confirmaram que **a entropia dos dados é o fator mais significativo** para determinar a eficácia dos algoritmos de compressão.

- **Arquivos de texto:** Baixa entropia → Alta compressão (67%)
- **Arquivos de imagem:** Entropia controlada → Boa compressão (71%)
- **Arquivos de áudio:** Alta variabilidade de entropia → Resultados variados

Implicação Prática

A seleção de algoritmos deve considerar as características estatísticas dos dados, não apenas o tipo de arquivo.

- **Conjunto de dados:** Limitado a três tipos de arquivo
- **Dados sintéticos:** Embora úteis para controle experimental, podem não representar totalmente cenários reais
- **Otimização:** Implementações poderiam ser otimizadas para performance (uso de cache, paralelização)
- **Arquivos comprimidos:** Não foi investigado o comportamento em arquivos já comprimidos

1 Extensão para outros formatos:

- ▶ JPEG, PNG (imagens)
- ▶ MP3, FLAC (áudio)
- ▶ Vídeo (H.264, H.265)

2 Algoritmos modernos:

- ▶ Brotli (Google)
- ▶ Zstandard (Facebook)
- ▶ Algoritmos baseados em aprendizado de máquina

3 Otimizações:

- ▶ Paralelização (multi-threading)
- ▶ Otimizações SIMD
- ▶ Comparação com implementações de referência

Obrigado!

Agradecimentos especiais:

Profa. Dra. Andréa Carla Gonçalves Vianna
Profa. Dra. Simone das Graças Domingues Prado
Profa. Dra. Juliana da Costa Feitosa
Universidade Estadual Paulista
Faculdade de Ciências - Bauru
Departamento de Computação

Código fonte disponível em:
<https://github.com/guhkadoo/unesp>

Referências I

-  Alakuijala, Jyrki et al. (2016). “Comparison of brotli, deflate, zopfli, LZMA, LZHAM and bzip2 compression algorithms”. In: *proceedings of the IEEE data compression conference*.
-  Collet, Yann and Chip Turner (2016). “Zstandard - Fast real-time compression algorithm”. In: *Proceedings of the IEEE Data Compression Conference*.
-  Deutsch, Peter (1996). *GZIP file format specification version 4.3*. Tech. rep. RFC 1952, May.
-  Hartley, Ralph VL (1928). “Transmission of information”. In: *Bell System technical journal* 7.3, pp. 535–563.
-  Huffman, David A (1952). “A method for the construction of minimum-redundancy codes”. In: *Proceedings of the IRE* 40.9, pp. 1098–1101.
-  Oord, Aaron van den et al. (2016). *WaveNet: A generative model for raw audio*. arXiv preprint arXiv:1609.03499.

Referências II

-  Salomon, David (2007). *Data compression: the complete reference*. Springer Science & Business Media.
-  Sapp, Craig Stuart (2024). *WAV Header Format*.
<http://soundfile.sapp.org/doc/WaveFormat/>. Acesso em: 03 nov. 2025.
-  Shannon, Claude Elwood (1948). “A mathematical theory of communication”. In: *The Bell system technical journal* 27.3, pp. 379–423.
-  Technology, Gdansk University of (2025). *BMP file format*.
<https://www.eg.bucknell.edu/~cs354/2016-fall/graphics-pipeline/bmp/bmp-file-format.html>. Acesso em: 03 nov. 2025.
-  Welch, Terry A (1984). “A technique for high-performance data compression”. In: *Computer* 17.6, pp. 8–19.



Ziv, Jacob and Abraham Lempel (1977). “A universal algorithm for sequential data compression”. In: *IEEE Transactions on information theory* 23.3, pp. 337–343.